Министерство науки и высшего образования РФ
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения задач»

Выполнили: ст. гр. 25BBB2
Денисов Р.С.
Никоноров С.Д.
Проверили: к.т.н., доцент
Деев М.В.
к.т.н., доцент
Юрова О.В.

Пенза 2026

Название: Бинарное дерево поиска

Цель работы: изучение структуры данных «бинарное дерево поиска», реализация основных операций над деревом (создание, вывод, поиск, подсчёт вхождений), а также оценка сложности процедуры поиска.

Лабораторные задания: 1) Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве; 2) Реализовать функцию подсчёта числа вхождений заданного элемента в дерево; 3) Изменить функцию добавления элементов для исключения добавления одинаковых символов; 4) Оценить сложность процедуры поиска по значению в бинарном дереве.

Теоретические сведения

Бинарное дерево — это структура данных, в которой каждый узел имеет не более двух потомков (левого и правого).

Бинарное дерево поиска (BST) — это бинарное дерево, обладающее дополнительными свойствами:

- значение левого потомка меньше значения родителя;
- значение правого потомка больше значения родителя.

Такие деревья используются для хранения данных в отсортированном виде и обеспечивают быстрый поиск.

Практическая часть

Псевдокод

Общие операции для всех структур

Создание нового элемента (get_struct)

```
ФУНКЦИЯ get_struct(data):
  ВЫДЕЛИТЬ память под узел r
  ЕСЛИ r = NULL ТО
    ВЫВЕСТИ "Ошибка выделения памяти"
    ЗАВЕРШИТЬ программу с кодом 0
  КОНЕЦ ЕСЛИ
  r.left  = NULL
```

```
  r.right = NULL
  r.data  = data
  ВЕРНУТЬ r
КОНЕЦ ФУНКЦИИ
```

Построение дерева с дубликатами (CreateTree)

```
ФУНКЦИЯ CreateTree(root, r, data):
  ЕСЛИ r = NULL ТО
    r = get_struct(data)
    ЕСЛИ root = NULL ТО
      ВЕРНУТЬ r
    КОНЕЦ ЕСЛИ
    ЕСЛИ data > root.data ТО
      root.left = r
    ИНАЧЕ
      root.right = r
    КОНЕЦ ЕСЛИ
    ВЕРНУТЬ r
  КОНЕЦ ЕСЛИ
  ЕСЛИ data > r.data ТО
    CreateTree(r, r.left, data)
  ИНАЧЕ
    CreateTree(r, r.right, data)
  КОНЕЦ ЕСЛИ
  ВЕРНУТЬ root
КОНЕЦ ФУНКЦИИ
```

Построение дерева без дубликатов (CreateTreeUnique)

```
ФУНКЦИЯ CreateTreeUnique(root, data):
  ЕСЛИ root = NULL ТО
    root = get_struct(data)
    ВЕРНУТЬ root
```

КОНЕЦ ЕСЛИ

ЕСЛИ data < root.data ТО

root.left = CreateTreeUnique(root.left, data)

ИНАЧЕ ЕСЛИ data > root.data ТО

root.right = CreateTreeUnique(root.right, data)

КОНЕЦ ЕСЛИ

ВЕРНУТЬ root

КОНЕЦ ФУНКЦИИ

Просмотр содержимого (print_tree)

```
ФУНКЦИЯ print_tree(r, l):
  ЕСЛИ r = NULL ТО
    ВЕРНУТЬСЯ
  КОНЕЦ ЕСЛИ
  print_tree(r.right, l + 1)
  ПОВТОРИТЬ l раз: ВЫВЕСТИ "    "
  ВЫВЕСТИ r.data
  print_tree(r.left, l + 1)
КОНЕЦ ФУНКЦИИ
```

Поиск значения (Search)

```
ФУНКЦИЯ Search(r, value):
  ЕСЛИ r = NULL ТО
    ВЕРНУТЬ NULL
  КОНЕЦ ЕСЛИ
  ЕСЛИ r.data = value ТО
    ВЕРНУТЬ r
  КОНЕЦ ЕСЛИ
  ЕСЛИ value < r.data ТО
    ВЕРНУТЬ Search(r.left, value)
  КОНЕЦ ЕСЛИ
  ВЕРНУТЬ Search(r.right, value)
```

КОНЕЦ ФУНКЦИИ

Подсчёт вхождений (CountOccurrences)

```
ФУНКЦИЯ CountOccurrences(r, value):
  ЕСЛИ r = NULL ТО
    ВЕРНУТЬ 0
  КОНЕЦ ЕСЛИ
  count = 0
  ЕСЛИ r.data = value ТО
    count = 1
  КОНЕЦ ЕСЛИ
  ВЕРНУТЬ count + CountOccurrences(r.left, value)
             + CountOccurrences(r.right, value)
КОНЕЦ ФУНКЦИИ
```

Освобождение памяти (FreeTree)

```
ФУНКЦИЯ FreeTree(r):
  ЕСЛИ r = NULL ТО
    ВЕРНУТЬСЯ
  КОНЕЦ ЕСЛИ
  FreeTree(r.left)
  FreeTree(r.right)
  ОСВОБОДИТЬ память r
КОНЕЦ ФУНКЦИИ
```

Главная программа (main)

```
ФУНКЦИЯ main:
  root = NULL
  start = 1
  ВЫВЕСТИ "-1 - окончание построения дерева"
  ПОКА start = 1:
    ВЫВЕСТИ "Введите число: "
    СЧИТАТЬ число D
```

```
  ЕСЛИ D = -1 ТО
    start = 0
    ПРЕРВАТЬ цикл
  КОНЕЦ ЕСЛИ
  root = CreateTree(root, root, D)
КОНЕЦ ПОКА
ВЫВЕСТИ "Построенное дерево:"
print_tree(root, 0)
ВЫВЕСТИ "Введите значение для поиска: "
СЧИТАТЬ value
found = Search(root, value)
ЕСЛИ found ≠ NULL ТО
  ВЫВЕСТИ "Значение ", value, " найдено (адрес: ", found, ")"
ИНАЧЕ
  ВЫВЕСТИ "Значение ", value, " не найдено"
КОНЕЦ ЕСЛИ
cnt = CountOccurrences(root, value)
ВЫВЕСТИ "Число вхождений значения ", value, ": ", cnt
FreeTree(root)
root = NULL
ВЫВЕСТИ "--- Построение дерева без дубликатов ---"
ВЫВЕСТИ "-1 - окончание построения дерева"
start = 1
ПОКА start = 1:
  ВЫВЕСТИ "Введите число: "
  СЧИТАТЬ число D
  ЕСЛИ D = -1 ТО
    start = 0
    ПРЕРВАТЬ цикл
  КОНЕЦ ЕСЛИ
```

```
    root = CreateTreeUnique(root, D)
  КОНЕЦ ПОКА
  ВЫВЕСТИ "Дерево без дубликатов:"
  print_tree(root, 0)
  FreeTree(root)
  ВЕРНУТЬ 0
КОНЕЦ ФУНКЦИИ
```

Задание 4. Оценка сложности

Скорость поиска в бинарном дереве поиска определяется его высотой — то есть числом переходов от корневого узла до наиболее удалённого листа.

Высота, в свою очередь, зависит от того, какую форму приняло дерево. В наилучшем варианте дерево оказывается сбалансированным: с ростом количества узлов высота увеличивается крайне медленно. Причина в том, что каждый новый уровень способен вместить вдвое больше узлов, чем предыдущий. Поэтому высота — это показатель степени, в которую нужно возвести двойку, чтобы получить общее число узлов n.

В среднем, когда данные вводятся в произвольном порядке, дерево получается достаточно разветвлённым, и сложность поиска оказывается примерно такой же, как в наилучшем случае.

Однако в наихудшем варианте дерево превращается в обычный линейный список. Это случается, когда данные поступают в уже упорядоченном виде.

Таким образом, бинарное дерево поиска обеспечивает быстрый поиск лишь до тех пор, пока сохраняет сбалансированность. Если элементы добавляются по возрастанию или убыванию, дерево вырождается, и все его преимущества пропадают.

Результат работы

```
-1 - окончание построения дерева
Введите число: 3
Введите число: 5
Введите число: 15
Введите число: 132
Введите число: 5
Введите число: 5
Введите число: 10
Введите число: -1

Дерево:
            132
        15
                    10
                5
            5
    5
3

Введите значение для поиска: 5
Значение 5 найдено
Число вхождений значения 5 в дерево: 3
```

```
Построение дерева без дубликатов
-1 - окончание построения дерева
Введите число: 1
Введите число: 1
Введите число: 5
Введите число: 23
Введите число: 1
Введите число: 5
Введите число: 6
Введите число: 7
Введите число: -1

Дерево без дубликатов:
        23
                7
            6
    5
1
```

Вывод

Освоили реализацию алгоритма поиска значения, которое вводится с клавиатуры, в уже построенном дереве. Также реализовали функцию, подсчитывающую, сколько раз заданный элемент встречается в дереве. Доработали функцию добавления элементов таким образом, чтобы одинаковые значения больше не попадали в дерево. Оценили сложность процедуры поиска по значению в бинарном дереве.